Data transformation with dplyr

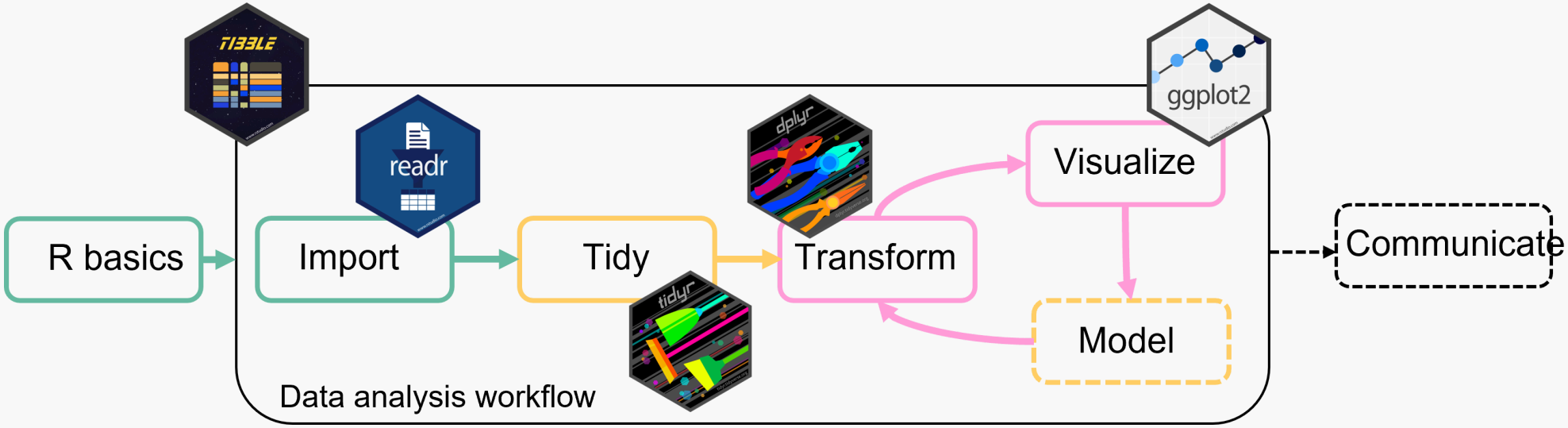
Day 2 - Introduction to Data Analysis with R

Selina Baldauf
Freie Universität Berlin - Theoretical Ecology

March 8, 2026

Data transformation

Data transformation is an important step in **understanding** the data and **preparing** it for further analysis.



We can use the tidyverse package `dplyr` for this.

Data transformation

With `dplyr` we can (among other things)

- **Filter** data to analyse only a part of it
- **Create** new variables
- **Summarize** data
- **Combine** multiple tables
- **Rename** variables
- **Reorder** observations or variables

To get started load the package `dplyr`:

```
library(dplyr)
# or
library(tidyverse)
```

Dplyr basic vocabulary

All of the `dplyr` functions work similarly:

- **First argument** is the data (a tibble)
- **Other arguments** specify what to do exactly
- **Return** a tibble

The data

The `gapminder` dataset contains data on life expectancy, GDP per capita, and population for 142 countries from 1952 to 2007.

```
# install.packages("gapminder")
library(gapminder)
gapminder
#> # A tibble: 1,704 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,694 more rows
```

`filter()`

> picks rows based on their value

filter()

Filter only European countries:

```
filter(gapminder, continent == "Europe")
#> # A tibble: 360 × 6
#>    country continent  year lifeExp     pop gdpPercap
#>    <fct>   <fct>     <int>   <dbl>   <int>     <dbl>
#>  1 Albania Europe     1952    55.2 1282697     1601.
#>  2 Albania Europe     1957    59.3 1476505     1942.
#>  3 Albania Europe     1962    64.8 1728137     2313.
#>  4 Albania Europe     1967    66.2 1984060     2760.
#>  5 Albania Europe     1972    67.7 2263554     3313.
#>  6 Albania Europe     1977    68.9 2509048     3533.
#>  7 Albania Europe     1982    70.4 2780097     3631.
#>  8 Albania Europe     1987    72   3075321     3739.
#>  9 Albania Europe     1992    71.6 3326498     2497.
#> 10 Albania Europe     1997    73.0 3428038     3193.
#> # ℹ 350 more rows
```

`filter()` goes through each row of the data and returns only those rows where the value for `continent` is `"Europe"`

filter()

You can also combine filters using logical operators (&, |, !):

```
filter(gapminder, continent == "Europe" & year == 2007)
#> # A tibble: 30 × 6
#>    country                continent  year lifeExp      pop gdpPercap
#>    <fct>                  <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Albania                Europe     2007    76.4  3600523     5937.
#>  2 Austria                Europe     2007    79.8  8199783    36126.
#>  3 Belgium                Europe     2007    79.4 10392226    33693.
#>  4 Bosnia and Herzegovina Europe     2007    74.9  4552198     7446.
#>  5 Bulgaria               Europe     2007    73.0  7322858    10681.
#>  6 Croatia                Europe     2007    75.7  4493312    14619.
#>  7 Czech Republic         Europe     2007    76.5 10228744    22833.
#>  8 Denmark                Europe     2007    78.3  5468120    35278.
#>  9 Finland                Europe     2007    79.3  5238460    33207.
#> 10 France                 Europe     2007    80.7 61083916    30470.
#> # ℹ 20 more rows
```

filter() + %in%

Use the %in% operator to filter rows based on multiple values, e.g. continents

```
my_continents <- c("Europe", "Africa", "Asia")
filter(gapminder, continent %in% my_continents)
#> # A tibble: 1,380 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1952    28.8  8425333      779.
#>  2 Afghanistan Asia       1957    30.3  9240934      821.
#>  3 Afghanistan Asia       1962    32.0 10267083      853.
#>  4 Afghanistan Asia       1967    34.0 11537966      836.
#>  5 Afghanistan Asia       1972    36.1 13079460      740.
#>  6 Afghanistan Asia       1977    38.4 14880372      786.
#>  7 Afghanistan Asia       1982    39.9 12881816      978.
#>  8 Afghanistan Asia       1987    40.8 13867957      852.
#>  9 Afghanistan Asia       1992    41.7 16317921      649.
#> 10 Afghanistan Asia       1997    41.8 22227415      635.
#> # ℹ 1,370 more rows
```

`filter()` + `is.na()`

You can use `is.na()` to check for missing values:

```
# Filter rows where lifeExp is missing
filter(gapminder, is.na(lifeExp))
```

Or keep only rows **without** missing values using `filter_out()`:

```
# Remove rows where lifeExp is missing
filter_out(gapminder, is.na(lifeExp))
```

The gapminder data has no missing values, so these filters don't change anything here. But this is very useful for real-world data!

`filter()` + `between()`

Filter rows where the value for `year` is between 1980 and 2000

```
filter(gapminder, between(year, 1980, 2000))
#> # A tibble: 568 × 6
#>    country     continent  year lifeExp      pop gdpPercap
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#>  1 Afghanistan Asia       1982    39.9 12881816      978.
#>  2 Afghanistan Asia       1987    40.8 13867957      852.
#>  3 Afghanistan Asia       1992    41.7 16317921      649.
#>  4 Afghanistan Asia       1997    41.8 22227415      635.
#>  5 Albania     Europe     1982    70.4  2780097     3631.
#>  6 Albania     Europe     1987    72    3075321     3739.
#>  7 Albania     Europe     1992    71.6  3326498     2497.
#>  8 Albania     Europe     1997    73.0  3428038     3193.
#>  9 Algeria     Africa     1982    61.4 20033753     5745.
#> 10 Algeria     Africa     1987    65.8 23254956     5681.
#> # ℹ 558 more rows
```

Or you could also do it like this:

```
filter(gapminder, year >= 1980 & year <= 2000)
```

Useful `filter()` helpers

These functions and operators help you filter your observations:

- relational operators `<`, `>`, `==`, …
- logical operators `&`, `|`, `!`
- `%in%` to filter multiple values
- `is.na()` to filter missing values
- `filter_out()` to remove rows matching a condition
- `between()` to filter values that are between an upper and lower boundary
- `near()` to compare floating points (use instead of `==` for doubles)

`select()`

> picks columns based on their names

select()

Select the columns `country`, `year`, and `lifeExp`

```
select(gapminder, country, year, lifeExp)
#> # A tibble: 1,704 × 3
#>    country      year lifeExp
#>    <fct>       <int>   <dbl>
#>  1 Afghanistan  1952    28.8
#>  2 Afghanistan  1957    30.3
#>  3 Afghanistan  1962    32.0
#>  4 Afghanistan  1967    34.0
#>  5 Afghanistan  1972    36.1
#>  6 Afghanistan  1977    38.4
#>  7 Afghanistan  1982    39.9
#>  8 Afghanistan  1987    40.8
#>  9 Afghanistan  1992    41.7
#> 10 Afghanistan  1997    41.8
#> # ℹ 1,694 more rows
```

Remove variables using `-`

```
select(gapminder, -country, -year, -lifeExp)
```

select() + starts_with()

Select all columns that start with "c"

```
select(gapminder, starts_with("c"))
```

```
#> # A tibble: 1,704 × 2
#>   country     continent
#>   <fct>       <fct>
#> 1 Afghanistan Asia
#> 2 Afghanistan Asia
#> 3 Afghanistan Asia
#> # ℹ 1,701 more rows
```

You can use the same structure for `ends_with()` and `contains()`.

```
# select columns that end with "Exp" or "cap"
select(gapminder, ends_with("Exp"))

# select columns that contain "life"
select(gapminder, contains("life"))
```

Useful `select()` helpers

- `starts_with()` and `ends_with()`: variable names that start/end with a specific string
- `contains()`: variable names that contain a specific string
- `matches()`: variable names that match a regular expression
- `any_of()` and `all_of()`: variables that are contained in a character vector

`mutate()`

> Adds new columns to your data

mutate()

New columns can be added based on values from other columns

```
mutate(gapminder, gdp = gdpPercap * pop)
```

```
#> # A tibble: 1,704 × 7
#>   country     continent  year lifeExp      pop gdpPercap         gdp
#>   <fct>       <fct>     <int>   <dbl>    <int>     <dbl>       <dbl>
#> 1 Afghanistan Asia       1952    28.8  8425333      779. 6567086330.
#> 2 Afghanistan Asia       1957    30.3  9240934      821. 7585448670.
#> 3 Afghanistan Asia       1962    32.0 10267083      853. 8758855797.
#> # ℹ 1,701 more rows
```

Add multiple new columns at once:

```
mutate(gapminder, gdp = gdpPercap * pop, pop_million = pop / 1e6)
```

mutate() + case_when()

Use `case_when` to add column values conditional on other columns.

`case_when()` can combine many cases into one.

```
mutate(
  gapminder,
  life_category = case_when(
    lifeExp < 50 ~ "low", # case 1
    lifeExp < 70 ~ "medium", # case 2
    lifeExp >= 70 ~ "high", # case 3
    .default = NA # all other
  )
)
#> # A tibble: 1,704 × 7
#>    country     continent  year lifeExp      pop gdpPercap life_category
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl> <chr>
#>  1 Afghanistan Asia       1952    28.8  8425333      779. low
#>  2 Afghanistan Asia       1957    30.3  9240934      821. low
#>  3 Afghanistan Asia       1962    32.0 10267083      853. low
#>  4 Afghanistan Asia       1967    34.0 11537966      836. low
#>  5 Afghanistan Asia       1972    36.1 13079460      740. low
#>  6 Afghanistan Asia       1977    38.4 14880372      786. low
#>  7 Afghanistan Asia       1982    39.9 12881816      978. low
#>  8 Afghanistan Asia       1987    40.8 13867957      852. low
#>  9 Afghanistan Asia       1992    41.7 16317921      649. low
#> 10 Afghanistan Asia       1997    41.8 22227415      635. low
#> # ℹ 1,694 more rows
```

`summarize()`

> summarizes data

summarize()

`summarize` will collapse the data to a single row

```
summarize(
  gapminder,
  mean_lifeExp = mean(lifeExp),
  mean_gdpPercap = mean(gdpPercap)
)
#> # A tibble: 1 × 2
#>   mean_lifeExp mean_gdpPercap
#>          <dbl>          <dbl>
#> 1         59.5          7215.
```

`summarize()` by group

`summarize` is much more useful in combination with the grouping argument `.by`

- **summary** will be calculated **separately for each group**

```
# summarize the grouped data
summarize(
  gapminder,
  mean_lifeExp = mean(lifeExp),
  mean_gdpPercap = mean(gdpPercap),
  .by = continent
)
#> # A tibble: 5 × 3
#>   continent mean_lifeExp mean_gdpPercap
#>   <fct>            <dbl>          <dbl>
#> 1 Asia              60.1          7902.
#> 2 Europe            71.9         14469.
#> 3 Africa            48.9          2194.
#> 4 Americas          64.7          7136.
#> 5 Oceania           74.3         18622.
```

- Combine variables if you want to summarize by more than one group (e.g. `.by = c(continent, year)`)

count()

Counts observations by group

```
# count rows grouped by continent
count(gapminder, continent)
#> # A tibble: 5 × 2
#>   continent     n
#>   <fct>     <int>
#> 1 Africa      624
#> 2 Americas    300
#> 3 Asia        396
#> 4 Europe      360
#> 5 Oceania      24
```

The pipe |>

Combine multiple data operations into one command

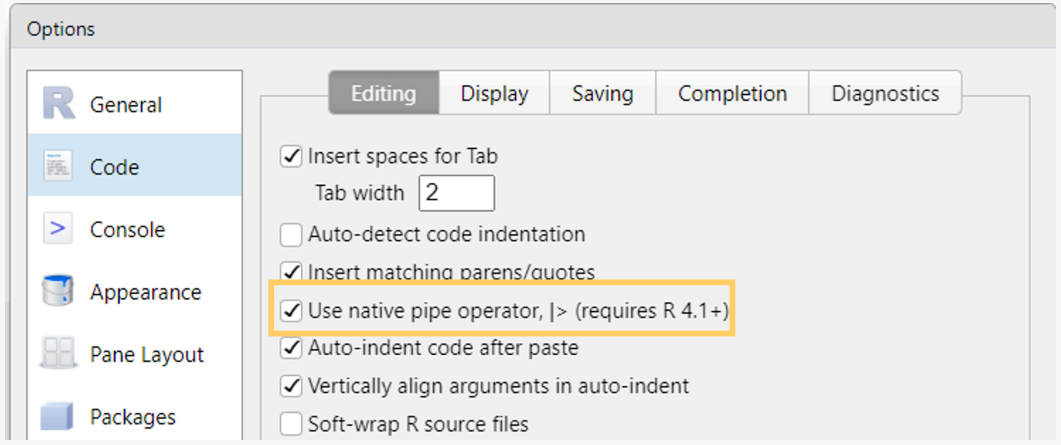
The pipe |>

Data transformation often requires **multiple operations** in sequence.

The pipe operator |> helps to keep these operations clear and readable.

- You may also see `%>%` from the `magrittr` package

Turn on the native R pipe |> in **Tools -> Global Options -> Code**



The pipe |>

Let's look at an example without pipe:

```
# 1: filter only data from 2007
gapminder_2007 <- filter(gapminder, year == 2007)

# 2: count countries by continent
gapminder_2007 <- count(gapminder_2007, continent)
```

How could we make this more efficient?

Use one **nested function** without intermediate results:

```
gapminder_2007 <- count(
  filter(gapminder, year == 2007),
  continent
)
```

But this gets complicated and error prone very quickly

The pipe |>

The pipe operator makes it very easy to combine multiple operations:

```
gapminder_2007 <- gapminder |>
  filter(year == 2007) |>
  count(continent)

gapminder_2007
```

You can read from top to bottom and interpret the |> as an "and then do".

The pipe |>

But what is happening?

The pipe is "pushing" the result of one line into the first argument of the function from the next line.

```
gapminder |>
  count(continent)

# instead of
count(gapminder, continent)
```

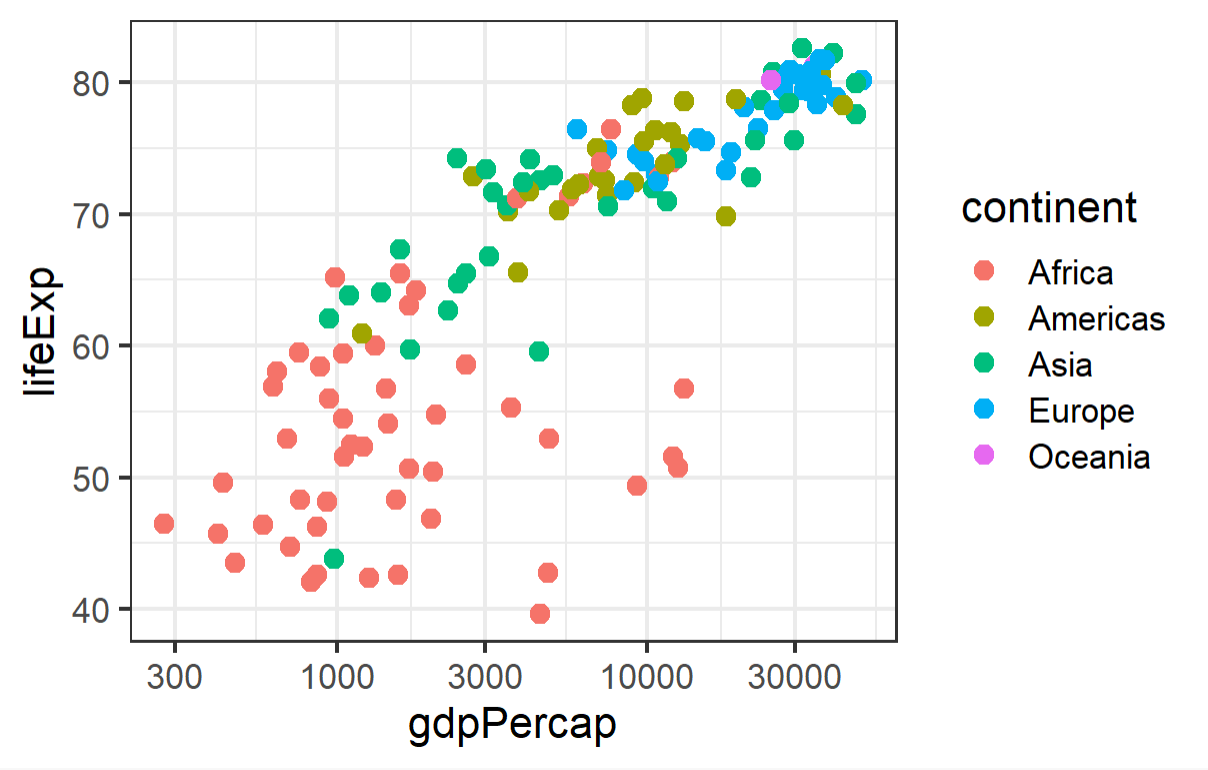
Piping works perfectly with the `tidyverse` functions because they are designed to return a tibble **and** take a tibble as first argument.

> Tip
>
> Use the keyboard shortcut `Ctrl/Cmd + Shift + M` to insert `|>`

The pipe |>

Piping also works well together with `ggplot`

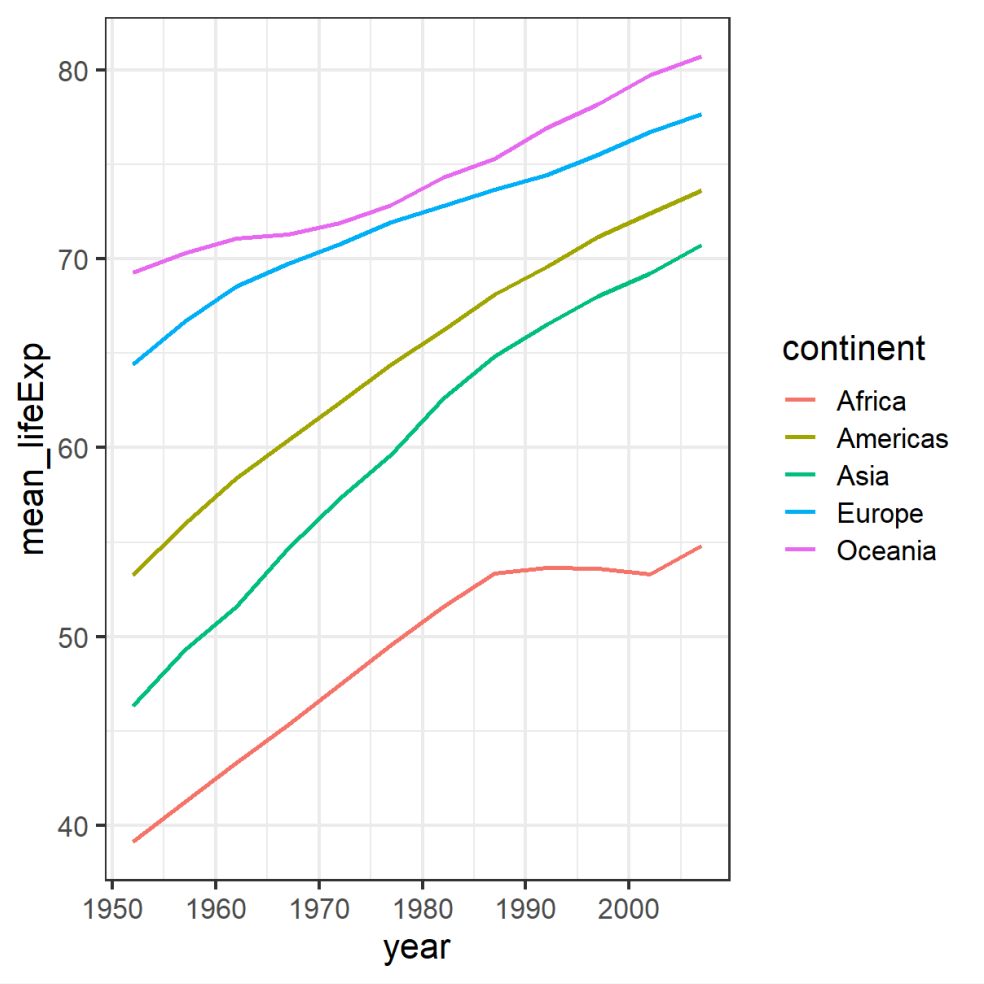
```
gapminder |>
  filter(year == 2007) |>
  ggplot(aes(x = gdpPercap, y = lifeExp, color = continent)) +
  geom_point(size = 3) +
  scale_x_log10()
```



The pipe |>

You can also combine dplyr operations and ggplot:

```
gapminder |>
  summarize(
    mean_lifeExp = mean(lifeExp),
    .by = c(continent, year)
  ) |>
  ggplot(
    aes(
      x = year,
      y = mean_lifeExp,
      color = continent
    )
  ) +
  geom_line(linewidth = 1)
```



- Use |> between dplyr steps
- Switch to + once you start the ggplot

Combining multiple tables

Combine two tibbles by row `bind_rows`

Situation: Two (or more) `tibbles` with the same variables (column names)

```
tbl_a <- gapminder[1:2, ] # first two rows
tbl_b <- gapminder[3:nrow(gapminder), ] # the rest
```

```
tbl_a
```

```
#> # A tibble: 2 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
```

```
tbl_b
```

```
#> # A tibble: 1,702 × 6
#>   country     continent  year lifeExp      pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#> 1 Afghanistan Asia       1962    32.0 10267083      853.
#> 2 Afghanistan Asia       1967    34.0 11537966      836.
#> # ℹ 1,700 more rows
```

Combine two tibbles by row `bind_rows`

Bind the rows together with `bind_rows()`:

```
bind_rows(tbl_a, tbl_b)
```

```
#> # A tibble: 1,704 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
#> # ℹ 1,702 more rows
```

You can also add an ID-column to indicate which line belonged to which table:

```
bind_rows(a = tbl_a, b = tbl_b, .id = "id")
```

```
#> # A tibble: 1,704 × 7
#>   id    country     continent  year lifeExp      pop gdpPercap
#>   <chr> <fct>       <fct>     <int>   <dbl>    <int>     <dbl>
#> 1 a     Afghanistan Asia       1952    28.8  8425333      779.
#> 2 a     Afghanistan Asia       1957    30.3  9240934      821.
#> 3 b     Afghanistan Asia       1962    32.0 10267083      853.
#> # ℹ 1,701 more rows
```

You can use `bind_rows()` to bind as many tables as you want:

```
bind_rows(a = tbl_a, b = tbl_b, c = tbl_c, ..., .id = "id")
```

Join tibbles with `left_join()`

Situation: Two tables that share some but not all columns.

```
gapminder
```

```
#> # A tibble: 1,704 × 6
#>   country     continent  year lifeExp     pop gdpPercap
#>   <fct>       <fct>     <int>   <dbl>   <int>     <dbl>
#> 1 Afghanistan Asia       1952    28.8 8425333      779.
#> 2 Afghanistan Asia       1957    30.3 9240934      821.
#> # ℹ 1,702 more rows
```

```
# table with more information on the continents
continent_info
#> # A tibble: 5 × 2
#>   continent region
#>   <fct>     <chr>
#> 1 Asia      Global East
#> 2 Europe    Western World
#> 3 Africa    Global South
#> 4 Americas  Western Hemisphere
#> 5 Oceania   Pacific
```

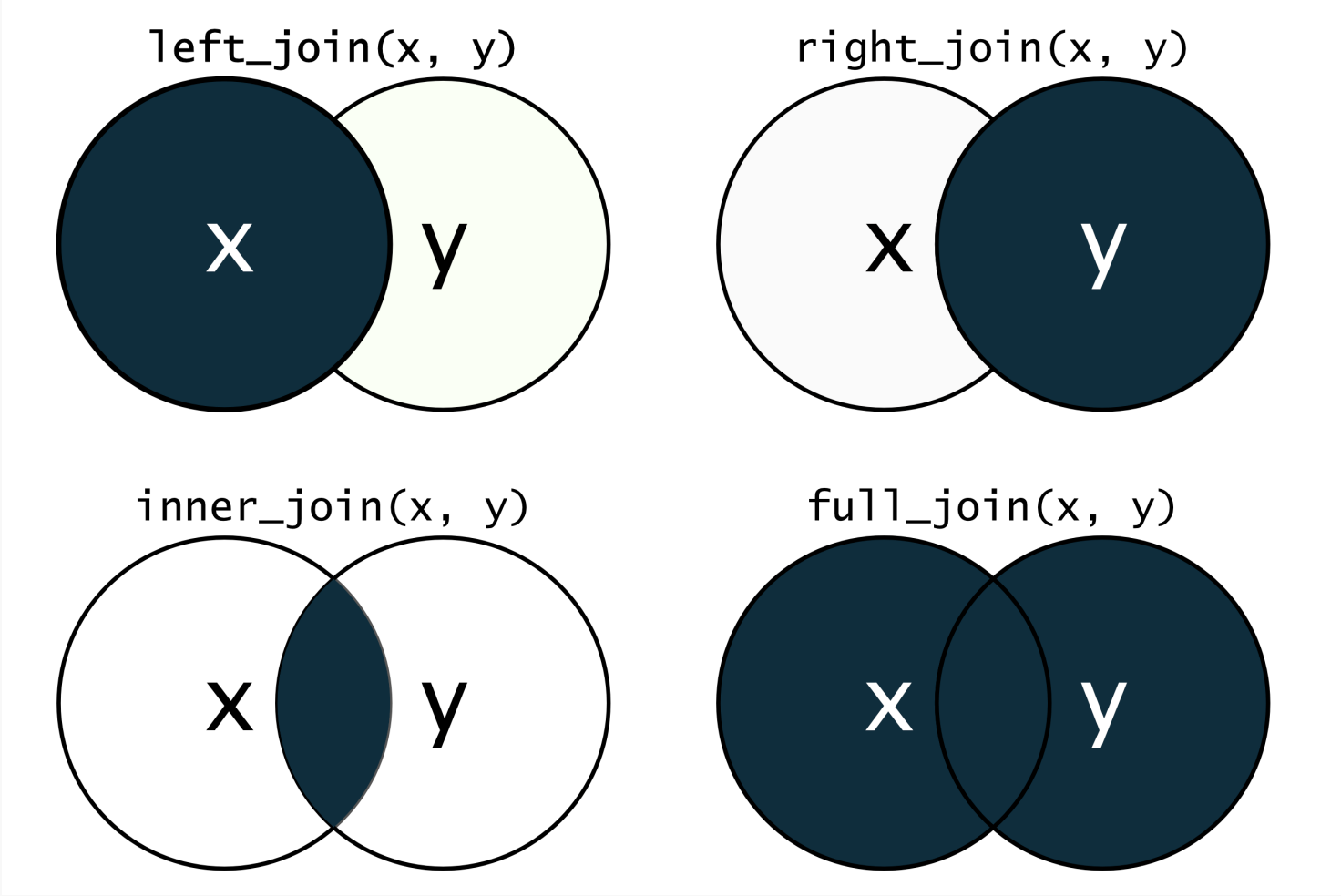
Join tibbles with `left_join()`

Join the two tables by the common column `continent`

```
left_join(gapminder, continent_info, by = "continent")
#> # A tibble: 1,704 × 7
#>    country     continent  year lifeExp      pop gdpPercap region
#>    <fct>       <fct>     <int>   <dbl>    <int>     <dbl> <chr>
#>  1 Afghanistan Asia       1952    28.8  8425333      779. Global East
#>  2 Afghanistan Asia       1957    30.3  9240934      821. Global East
#>  3 Afghanistan Asia       1962    32.0 10267083      853. Global East
#>  4 Afghanistan Asia       1967    34.0 11537966      836. Global East
#>  5 Afghanistan Asia       1972    36.1 13079460      740. Global East
#>  6 Afghanistan Asia       1977    38.4 14880372      786. Global East
#>  7 Afghanistan Asia       1982    39.9 12881816      978. Global East
#>  8 Afghanistan Asia       1987    40.8 13867957      852. Global East
#>  9 Afghanistan Asia       1992    41.7 16317921      649. Global East
#> 10 Afghanistan Asia       1997    41.8 22227415      635. Global East
#> # ℹ 1,694 more rows
```

`left_join()` means that the resulting tibble will contain all rows of `gapminder`, but not necessarily all rows of `continent_info` (in this case it does though).

Different `*_join()` functions



Summary

Data transformation with dplyr

Summary I

All `dplyr` functions take a tibble as first argument and return a tibble.

`filter()`

- **pick rows** with helpers
  - relational and logical operators
  - `%in%`
  - `is.na()` and `filter_out()`
  - `between()`
  - `near()`

Summary II

All `dplyr` functions take a tibble as first argument and return a tibble.

`select()`

- pick columns with helpers
  - `starts_with()`, `ends_with()`
  - `contains()`
  - `matches()`
  - `any_of()`, `all_of()`

Summary III

`arrange()`

- **change order** of rows (adscending)
  - or descending with `desc()`

`mutate()`

- **add columns** but keep all columns
  - `case_when()` for conditional values

Summary IV

`summarize()`

- **collapse rows** into one row by some summary
  - use `.by` argument to summarize by group

`count`

- **count rows** based on a group

Summary V

`bind_rows()`

- **combine rows** of multiple tibbles into one
  - the tibbles need to have the same columns
  - add an id column with the argument `.id = "id"`
  - function `bind_cols()` works similarly just for columns

`left_join()`

- **combine tables** based on common columns

Now you

Task (30 min)

Transform the penguin data set

Find the task description here